

Table of Contents

Case Study 1: Linux Memory Management and Virtual Memory.....	1
1.1 Introduction.....	1
1.2 What is Memory Management?.....	1
1.3 Why Memory Management is Important.....	1
1.4 Memory Management in Linux	1
1.5 Physical Memory and Virtual Memory	1
1.6 Linux Virtual Memory.....	2
1.7 Pages and Page Frames.....	2
1.8 Page Tables.....	2
1.9 Address Translation	3
1.10 Demand Paging.....	3
1.11 Page Faults.....	4
1.12 Memory Allocation in Linux	4
1.13 Kernel Memory Management.....	4
1.14 Swap Space.....	4
1.15 Memory Protection	4
1.16 Advantages of Linux Memory Management	5
1.17 Challenges.....	5
1.18 Real-World Example	5
1.19 Summary.....	5
Case Study 2: Real-World File Systems: NTFS, ext4 and HFS+	6
2.1 Introduction.....	6
2.2 What is a File System?.....	6
2.3 Why File Systems are Needed	6
2.4 How File Systems Store Data	6
2.5 NTFS.....	6
2.6 Features of NTFS.....	7
2.7 NTFS Journaling.....	7
2.8 NTFS Security and Permissions	7
2.9 ext4	7
2.10 Features of ext4.....	7
2.11 ext4 Journaling.....	8
2.12 ext4 File Allocation	8
2.13 HFS+.....	8

2.14 Features of HFS+	8
2.15 HFS+ Journaling	9
2.16 Comparison of NTFS, ext4 and HFS+	9
2.17 Advantages and Disadvantages.....	10
2.18 Real-World Uses.....	10
2.19 Summary.....	10
Case Study 3: The WannaCry Ransomware Attack	11
3.1 Introduction.....	11
3.2 What is Ransomware?	11
3.3 What was WannaCry?.....	11
3.4 Background of the Attack	11
3.5 The Windows Vulnerability.....	11
3.6 How WannaCry Spread	12
3.7 Role of the SMB Protocol.....	12
3.8 What Happened During the Attack.....	13
3.9 Timeline of the Attack	13
3.10 Systems Affected	13
3.11 Impact on Organizations	14
3.12 Impact on Healthcare	14
3.13 Impact on Businesses.....	14
3.14 The WannaCry Kill Switch.....	14
3.15 Microsoft Security Patch.....	14
3.16 Lessons Learned	14
3.17 How Organizations Can Prevent Similar Attacks.....	15
3.18 Importance of Operating System Updates	15
3.19 Summary.....	15
Case Study 4: Distributed Systems - GFS and Hadoop	16
4.1 Introduction.....	16
4.2 What is a Distributed System?	16
4.3 Why Distributed Systems are Used	16
4.4 Google File System (GFS).....	16
4.5 Why Google Developed GFS	16
4.6 GFS Architecture	16
4.7 GFS Master.....	17
4.8 GFS Chunkservers	17
4.9 Data Replication	18
4.10 Fault Tolerance	18

4.11 Hadoop.....	18
4.12 Hadoop Distributed File System (HDFS)	18
4.13 HDFS Architecture	19
4.14 NameNode	19
4.15 DataNodes.....	19
4.16 Data Replication in HDFS	19
4.17 Hadoop MapReduce	19
4.18 Fault Tolerance in Hadoop.....	20
4.19 Real-World Applications	20
4.20 GFS vs Hadoop.....	20
4.21 Advantages.....	21
4.22 Challenges.....	21
4.23 Summary.....	21

Case Study 1: Linux Memory Management and Virtual Memory

1.1 Introduction

Whenever a program runs, it needs memory for its data and instructions. Linux does not give out RAM randomly. The kernel keeps track of memory, protects it, and shares it among running programs. In this case study, I explain how Linux memory management works and how virtual memory helps a system run more programs than the available RAM might suggest.

1.2 What is Memory Management?

Memory management means controlling how RAM is given to programs, used, and released. When a program no longer needs a particular area, Linux can return that memory for other work. Good management is important because one program should not be able to consume all available RAM and affect the whole system.

1.3 Why Memory Management is Important

A computer often runs many programs at the same time, while RAM is limited. Linux therefore has to use memory efficiently, keep processes separated, and reclaim space when it is free again. These tasks improve both system stability and security.

1.4 Memory Management in Linux

Linux handles memory through a mix of hardware support and kernel software. The processor itself includes a piece of hardware called the Memory Management Unit (MMU), which Linux uses to keep each running program in its own private memory area. This means a bug or crash in one app, like a browser tab, normally cannot corrupt the memory of another app, like a music player.

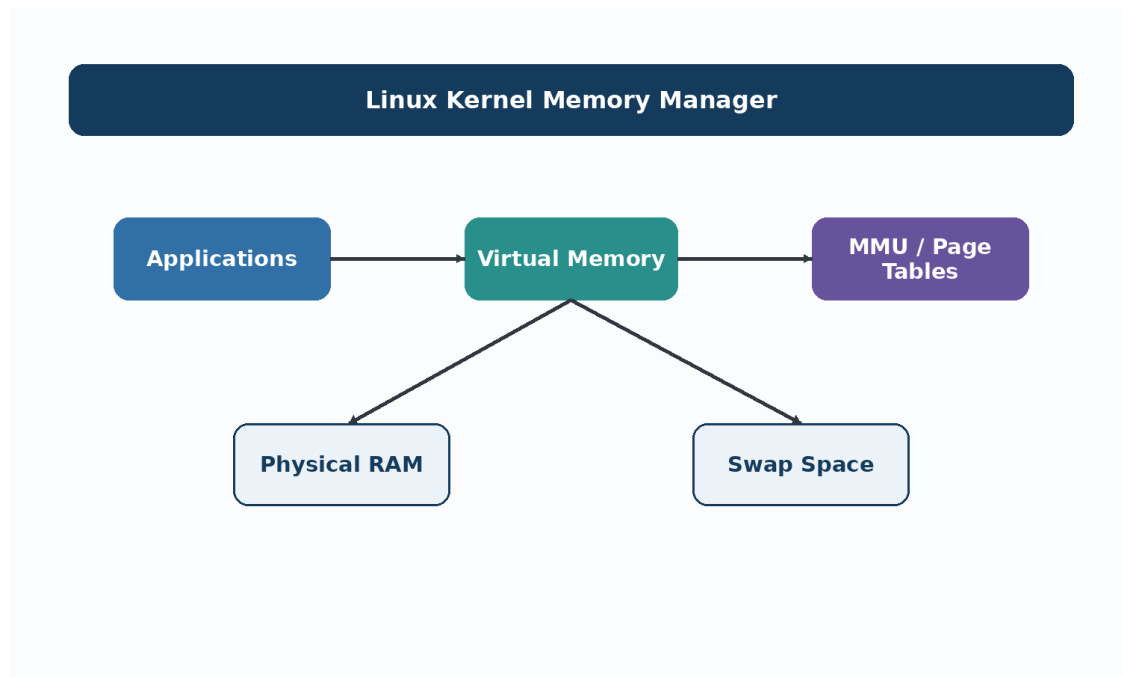


Figure 1. Basic Linux Memory Management Structure

1.5 Physical Memory and Virtual Memory

Physical memory is the real RAM installed in the computer, for example 8GB or 16GB. Virtual memory is a logical memory space created by the operating system. It gives each process its own address space even though the physical RAM is shared among many programs.

Here is a simple example. Suppose a laptop has 8GB of RAM, but ten programs are open, each acting as if it has access to a few gigabytes of memory. Linux does not actually give each program that much real RAM. Instead, it gives each one a virtual address space, and only the parts that are actively being used get placed into real RAM. The rest can be kept on disk until it is needed.

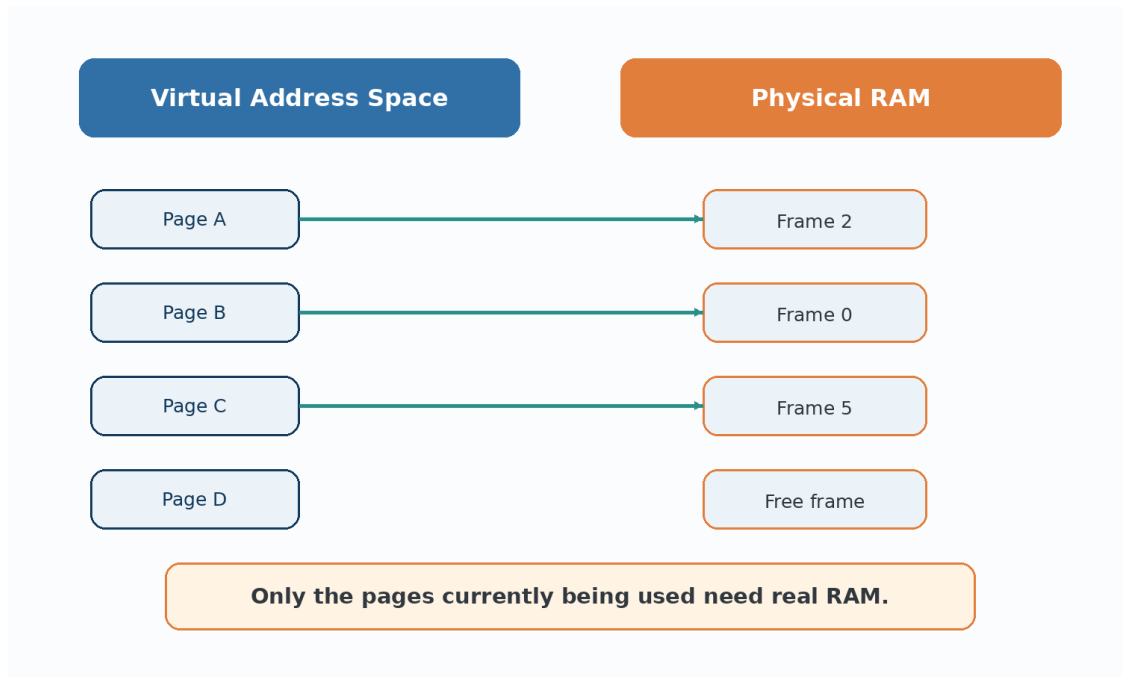


Figure 2. Physical Memory vs Virtual Memory in Linux

1.6 Linux Virtual Memory

In Linux, every process gets its own virtual address space, usually spanning from address zero up to a very large number, even though the actual RAM installed is far smaller. The operating system maps small pieces of this virtual space onto real physical memory only when needed. This mapping is invisible to the program itself. As far as the program can tell, its memory is all there, ready to use.

1.7 Pages and Page Frames

Rather than tracking memory byte by byte, Linux splits memory into fixed-size chunks called pages, which are commonly 4 kilobytes each. Virtual memory is divided into virtual pages, and physical RAM is divided into matching physical frames. A page from a program's virtual memory is placed into any free frame of physical RAM, and the operating system keeps a record of which virtual page is sitting in which physical frame.

1.8 Page Tables

A page table keeps the mapping between a process's virtual pages and the physical frames where those pages are stored. During memory access, the processor uses this mapping to find the required location in RAM.

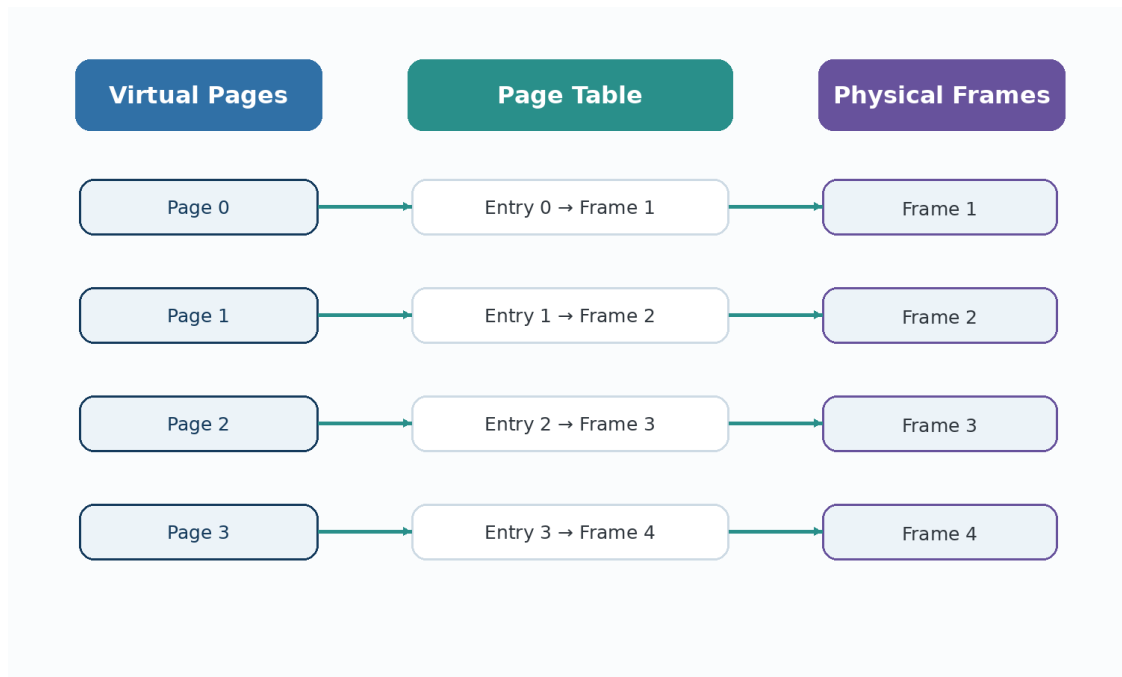


Figure 3. How a Page Table Maps Virtual Pages to Physical Frames

1.9 Address Translation

Address translation converts a virtual address into a physical address. The page number is used to find the correct frame in the page table, while the offset identifies the exact position inside that page. The MMU performs this work automatically during memory access.

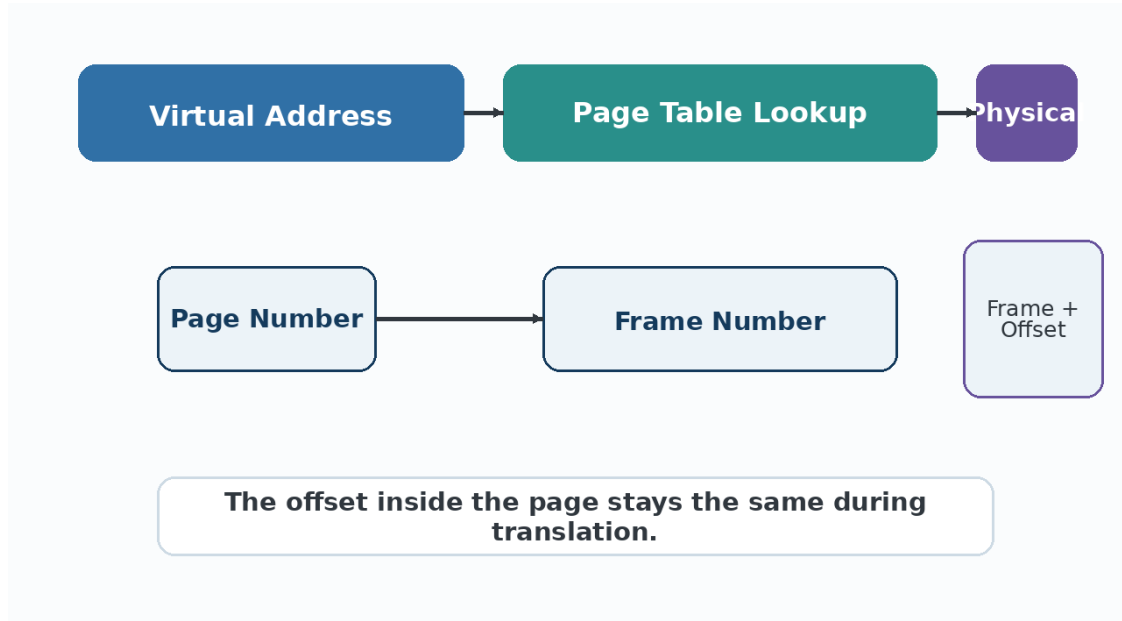


Figure 4. Simplified Virtual Address Translation in Linux

1.10 Demand Paging

Linux normally loads memory pages when they are needed rather than loading the whole program at startup. This is called demand paging. It saves RAM and allows large applications to start without immediately occupying all of their memory.

1.11 Page Faults

A page fault occurs when a requested page is not currently in RAM. Linux temporarily stops that process, locates the page, loads it into a free frame, updates the page table, and then allows the program to continue. So, a page fault is not always an error; it is often part of normal virtual-memory operation.

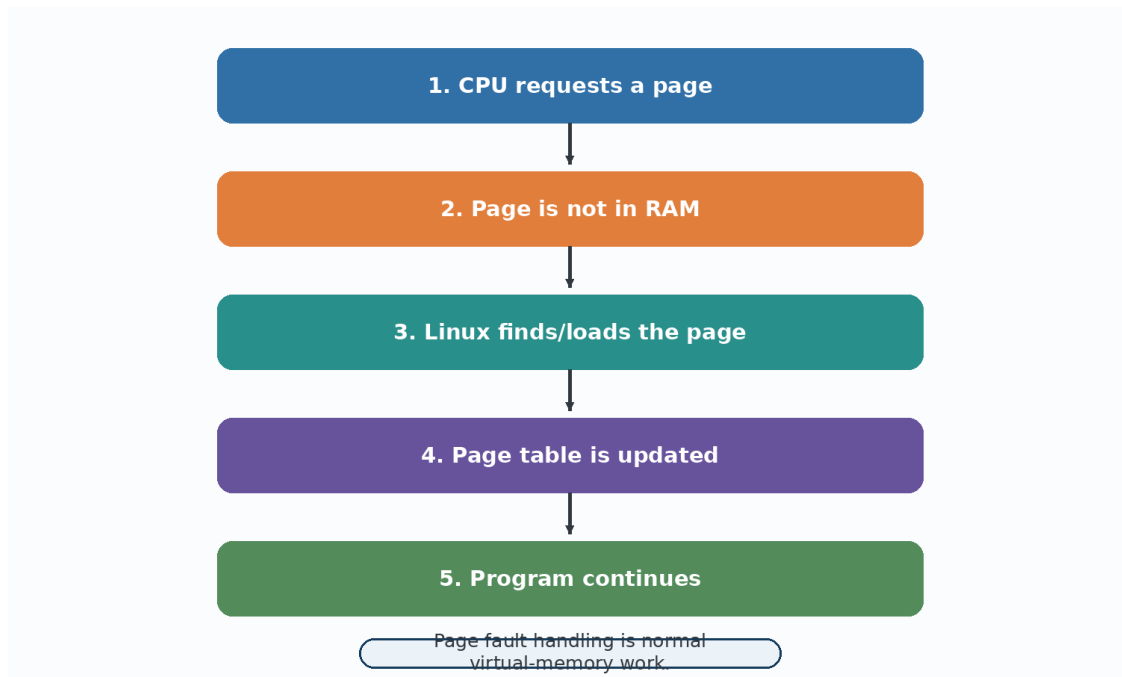


Figure 5. Steps Involved in Handling a Page Fault

1.12 Memory Allocation in Linux

When a program asks for more memory, for example when you open a new document in a text editor, Linux has to find a suitable free area and hand it over. Linux uses different allocation strategies for different needs: large chunks for whole processes, and smaller chunks handled through internal kernel allocators for things like file buffers and network data. This flexible approach keeps memory use efficient across many different kinds of programs running at once.

1.13 Kernel Memory Management

The Linux kernel itself, the core part of the operating system, also needs memory to keep track of processes, files, and devices. This kernel memory is kept in a separate kernel space, apart from the user space memory used by regular apps, which stops a bug in an ordinary app from directly damaging the kernel's own data.

1.14 Swap Space

Sometimes RAM fills up completely. When this happens, Linux can move pages that have not been used recently out of RAM and onto a special area of the disk called swap space. This frees up real RAM for pages that are needed right now. Swap acts like a backup memory area, though it is much slower than RAM since disks are far slower than memory chips, even fast SSDs.

1.15 Memory Protection

Memory protection stops one program from reading or changing another program's memory without permission. Since each process only has access to its own virtual address space, and the MMU enforces this at the hardware level, a crash in one app usually cannot bring down the whole system or leak another program's private data.

1.16 Advantages of Linux Memory Management

Programs can run even when their combined memory needs are larger than the physical RAM installed.

Each process is isolated, which improves stability and security.

Demand paging speeds up how quickly large programs start.

Swap space gives the system a safety net when RAM runs low.

1.17 Challenges

Too much reliance on swap space can slow the whole system down noticeably.

Managing memory fairly across many processes takes constant background work.

Memory leaks in badly written programs can still slowly use up available RAM.

Balancing performance and fairness between processes is a difficult ongoing design problem.

1.18 Real-World Example

For example, imagine an 8GB laptop running a video editor, a browser with many tabs, and a music application. Linux can keep actively used data in RAM while moving less-used pages to swap when necessary. The user can continue working even though all the program data cannot fit into RAM at the same time.

1.19 Summary

In summary, Linux combines virtual memory, pages, page tables, demand paging, and swap space to manage limited RAM. These mechanisms help processes run safely and allow the system to continue working even when memory demand becomes high.

Case Study 2: Real-World File Systems: NTFS, ext4 and HFS+

2.1 Introduction

A computer needs a clear method for storing and finding files on a disk. That method is provided by a file system. This case study compares NTFS used by Windows, ext4 used by Linux, and HFS+ from older macOS systems.

2.2 What is a File System?

A file system is the structure used by an operating system to store, organize, and retrieve files. It records information such as file names, sizes, locations, and access permissions. Without it, the storage device would simply be a large collection of bytes with no useful organization.

2.3 Why File Systems are Needed

A simple way to understand a file system is to compare it with a library. Shelves and a catalog help a person locate one book among thousands. In the same way, a file system keeps track of where files are stored and how they can be accessed.

2.4 How File Systems Store Data

Most file systems break a disk into small fixed-size blocks, and files are stored across one or more of these blocks. A separate area of the disk keeps track of metadata, meaning information about the files, such as their name, size, and which blocks hold their data. When a program asks to open a file, the file system looks up this metadata first, then goes straight to the correct blocks on disk.

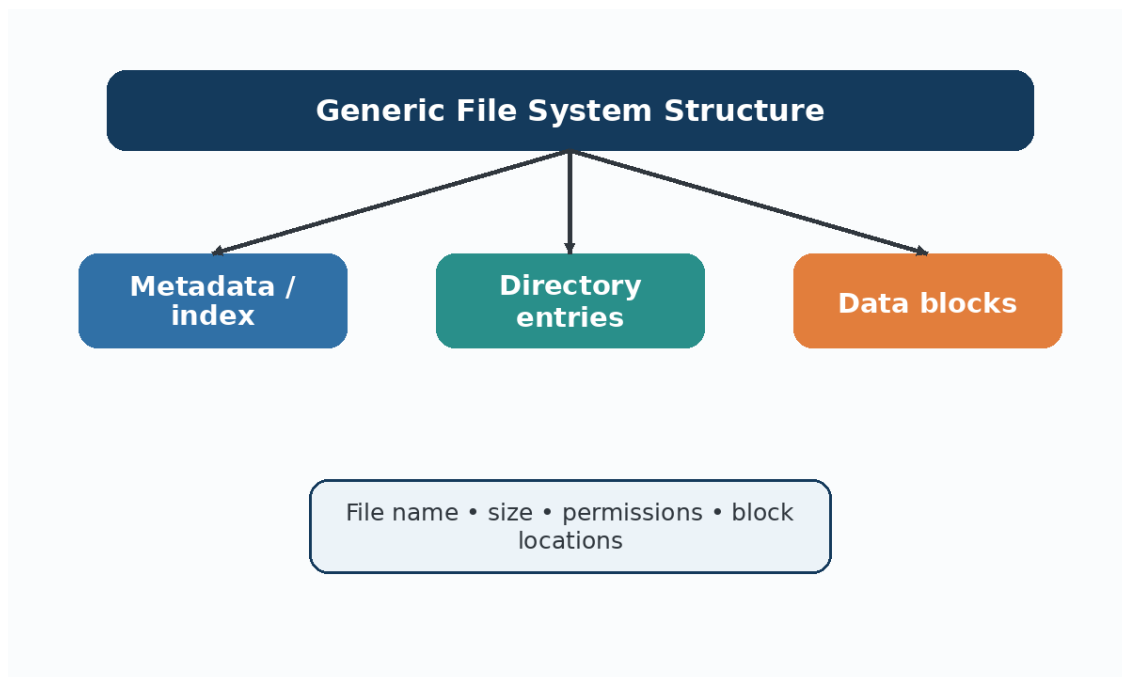


Figure 6. General Structure of a File System

2.5 NTFS

NTFS, short for New Technology File System, is the main file system used by Windows. It was designed to be more reliable and secure than the older FAT file systems, and it is still the default choice on Windows computers today.

2.6 Features of NTFS

Supports very large files and large disk volumes.

Keeps detailed permissions on who can read, write, or delete each file.

Uses a Master File Table (MFT) to store metadata about every file on the drive.

Supports file compression and encryption built into the file system.

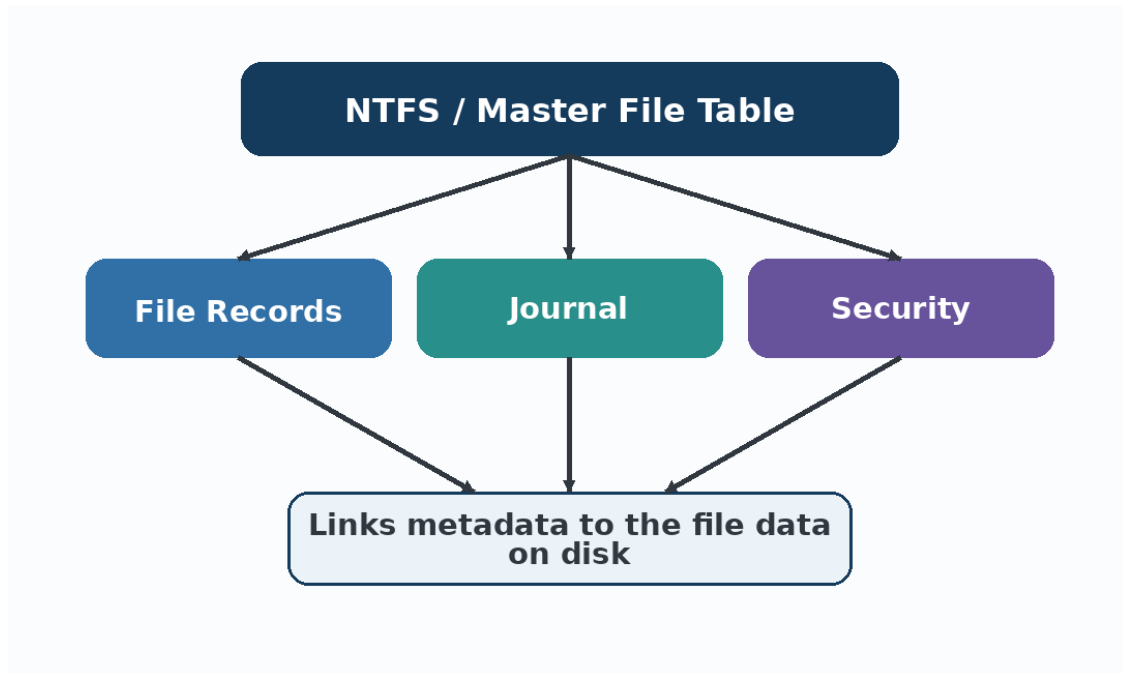


Figure 7. NTFS Architecture Showing the Master File Table

2.7 NTFS Journaling

NTFS uses journaling to record important disk changes before they are completed. If a crash or power failure happens, the journal helps the system recover more safely and reduces the chance of file-system corruption.

2.8 NTFS Security and Permissions

NTFS allows very fine control over who can access a file. A single file can have different permissions for different users or groups, such as allowing one person to only read a document while another person can edit it. This is one reason NTFS is popular in offices and organisations that need to control access to shared files.

2.9 ext4

ext4, short for the fourth extended file system, is one of the most widely used file systems on Linux. It replaced the older ext3 and ext2 file systems, adding better performance and support for much larger files and drives.

2.10 Features of ext4

Supports extremely large individual files, well beyond what most users will ever need.

Uses a feature called extents, which tracks large blocks of continuous disk space more efficiently than older methods.

Includes journaling for better crash recovery.

Is free and open source, matching the open nature of Linux itself.

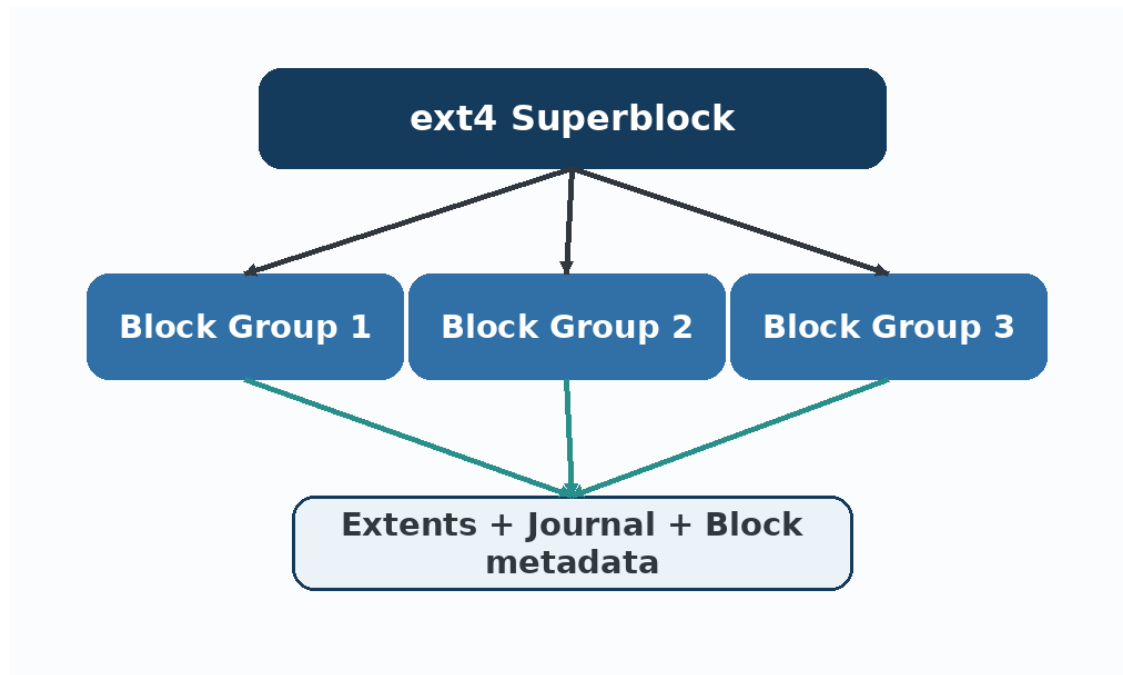


Figure 8. ext4 File System Structure with Block Groups

2.11 ext4 Journaling

ext4 also uses journaling. The journal records pending changes and helps the system return to a consistent state after an unexpected shutdown.

2.12 ext4 File Allocation

ext4 divides the disk into block groups, and files are stored using extents, which are groups of consecutive blocks recorded as a single entry instead of tracking every block one by one. This reduces the amount of bookkeeping needed for large files and makes reading big files faster.

2.13 HFS+

HFS+, short for Hierarchical File System Plus, was the main file system used by Apple's macOS for many years before being replaced by APFS on newer systems. It is still found on many older Mac computers and some external drives formatted for compatibility.

2.14 Features of HFS+

Organizes files using a catalog file that tracks the entire folder and file structure.

Supports file names in different languages and character sets.

Includes journaling for improved reliability, added in later versions.

Was designed with Mac-specific features like resource forks for storing extra file information.

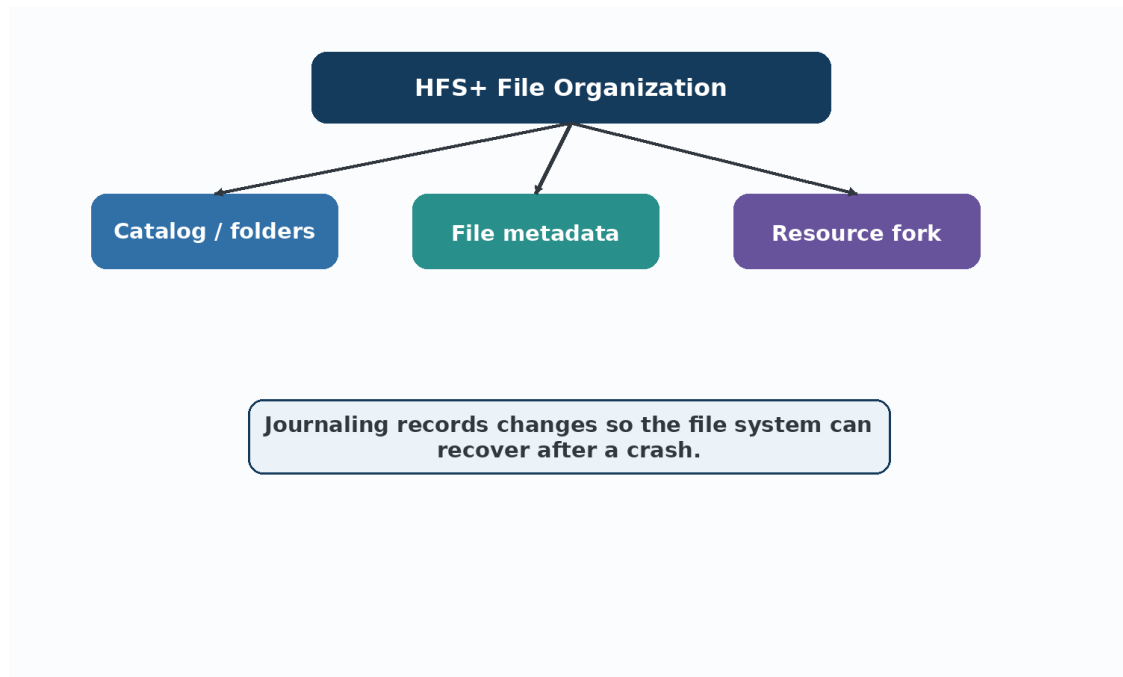


Figure 9. HFS+ File System Structure

2.15 HFS+ Journaling

Later versions of HFS+ included journaling. The basic idea is to record planned changes so the file system has a better chance of recovering after a crash or sudden power loss.

2.16 Comparison of NTFS, ext4 and HFS+

The following table gives a simple comparison of the three file systems and their main characteristics.

Feature	NTFS	ext4	HFS+
Operating System	Windows	Linux	Older macOS
Maximum File Size	Very large (16TB+)	Very large (16TB+)	Large (8EB in theory)
Journaling	Yes	Yes	Yes (added later)
Security	Strong, detailed permissions	Standard Linux permissions	Basic macOS permissions
Performance	Good for everyday use	Very good, especially for large files	Good, but older than the other two
Reliability	High	High	Moderate to high
Typical Usage	Windows PCs, external drives	Linux servers and desktops	Older Mac computers
Compatibility	Readable on most systems with extra tools	Mainly Linux, limited elsewhere	Mainly macOS, limited elsewhere

Table 3. Comparison of Google GFS and Hadoop

2.17 Advantages and Disadvantages

NTFS is closely integrated with Windows and provides detailed permissions. ext4 is a common Linux choice with good performance and open-source support. HFS+ was important for older Mac systems, but newer macOS versions use APFS instead.

2.18 Real-World Uses

In practice, NTFS is common on Windows office computers, ext4 is widely seen on Linux desktops and servers, and HFS+ may still appear on older Macs or compatible external drives. The actual choice depends on the operating system and the required features.

2.19 Summary

Overall, the three file systems solve the same basic storage problem but were designed for different environments. Their features, permissions, performance choices, and compatibility are closely connected to the operating systems they were created for.

Case Study 3: The WannaCry Ransomware Attack

3.1 Introduction

WannaCry was a major ransomware incident that spread quickly in May 2017. It disrupted hospitals, businesses, and public services in many countries. This case study focuses on what happened, why the malware spread so quickly, and what organizations learned about system security and updates.

3.2 What is Ransomware?

Ransomware is malware that blocks access to files or systems and demands payment from the victim. Because important data may become unavailable, a ransomware incident can interrupt normal business or public services.

3.3 What was WannaCry?

WannaCry mainly targeted older or unpatched Windows systems. After infection, it encrypted files and displayed a ransom message. Its major feature was automatic network spread, which allowed it to move from one vulnerable computer to another without repeated user interaction.

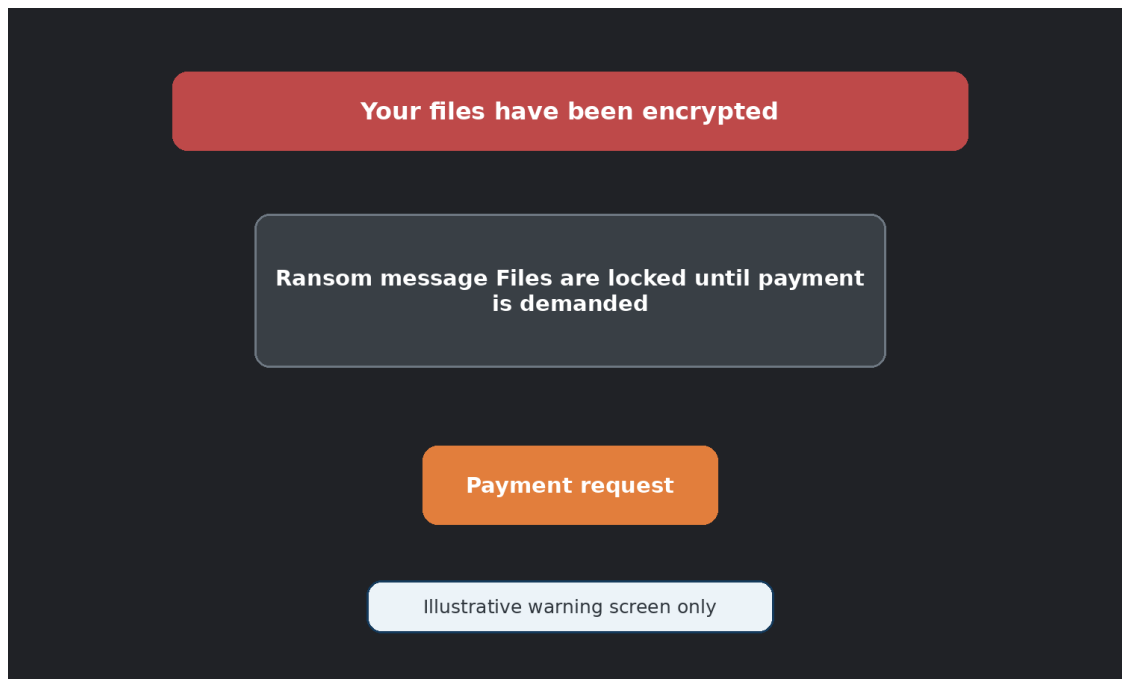


Figure 10. Illustrative Example of a Ransomware Warning Screen

3.4 Background of the Attack

The malware used a Windows vulnerability that had already been identified, and Microsoft had released a fix before the outbreak. However, many systems had not installed the update, so the known weakness remained open.

3.5 The Windows Vulnerability

The flaw WannaCry took advantage of was in a part of Windows used for sharing files and printers over a network. This kind of flaw is dangerous because it does not require a user to open a suspicious email or download a bad file. If a vulnerable computer was simply connected to a network with an infected machine, it could be attacked automatically.

3.6 How WannaCry Spread

After reaching one vulnerable computer, WannaCry searched the surrounding network for other systems with the same weakness. It could then spread automatically, which helped the infection move through some organizations very quickly.

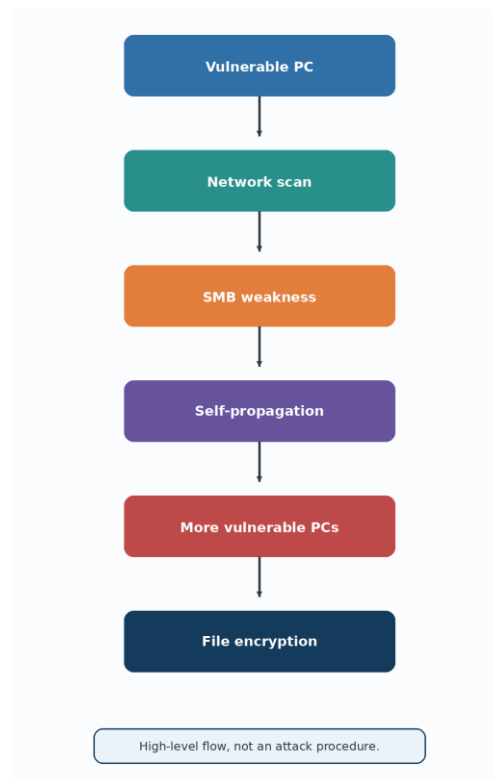


Figure 11. Simplified Attack Flow of the WannaCry Ransomware

3.7 Role of the SMB Protocol

The specific flaw used by WannaCry was in the Server Message Block (SMB) protocol, which Windows computers use to share files and printers on a local network. Because SMB is so commonly used in offices, the flaw gave WannaCry a wide path to spread quickly once it got inside a network.

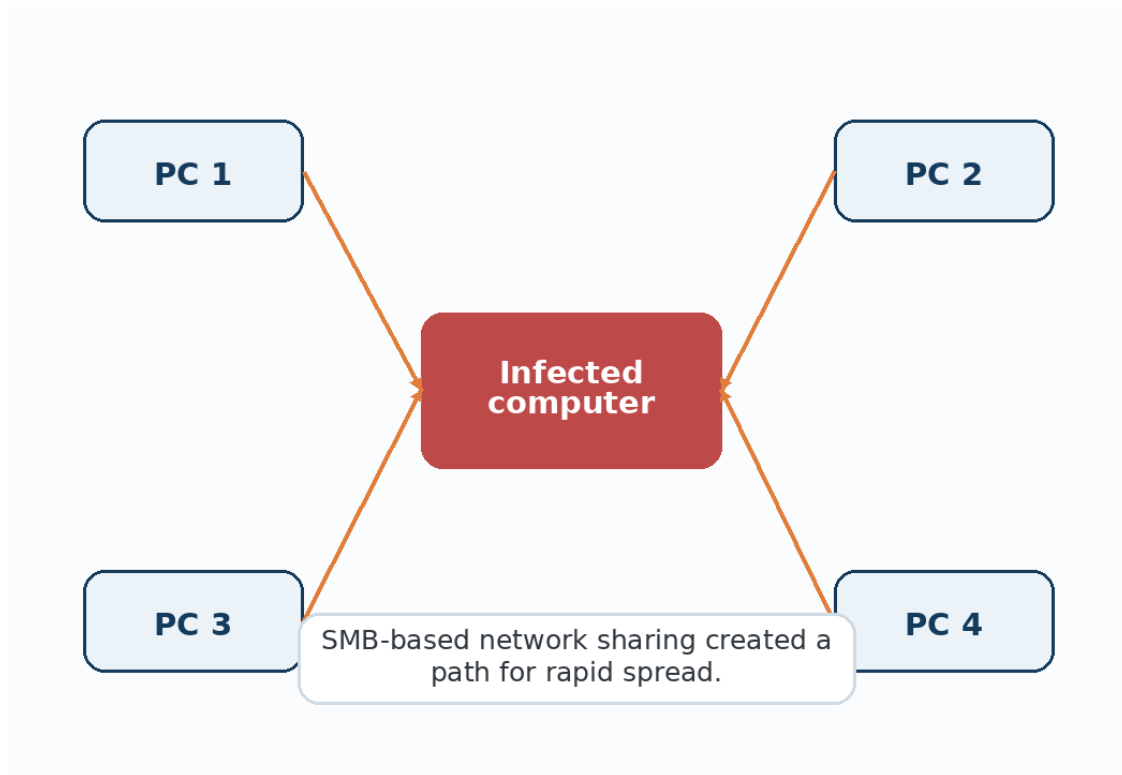


Figure 12. How an Infection Could Spread Across a Network Using File-Sharing Protocols

3.8 What Happened During the Attack

On 12 May 2017, the outbreak accelerated and reached systems around the world in a short period. Infected computers displayed a ransom message and a countdown, showing that files had been encrypted.

3.9 Timeline of the Attack

Date	Event
March 2017	Microsoft releases a security patch (MS17-010) fixing the SMB flaw.
April 2017	A tool that could exploit the flaw becomes publicly available online.
12 May 2017	WannaCry begins spreading rapidly across the world.
12 May 2017	A security researcher discovers and activates a hidden kill switch, slowing the spread.

3.10 Systems Affected

WannaCry mainly affected computers running older versions of Windows that had not installed the March 2017 security update. This included many computers in large organisations, where updates are sometimes delayed to first test compatibility with existing software.

3.11 Impact on Organizations

Available estimates put the impact at roughly 200,000 computers in more than 150 countries. The wider financial cost included downtime, recovery work, and business disruption and was estimated in the hundreds of millions of dollars.

3.12 Impact on Healthcare

The UK's National Health Service experienced serious disruption because some systems became unavailable. Some hospitals and clinics had to cancel appointments and reduce normal services while affected computers were restored.

3.13 Impact on Businesses

Businesses in sectors such as shipping, manufacturing, and telecommunications also reported service interruptions. In some cases, systems and production activities were stopped while organizations cleaned infected machines and restored services.

3.14 The WannaCry Kill Switch

A security researcher discovered that the malware was checked for a particular web address. Registering that address activated a built-in condition that stopped further spreading in many cases and significantly reduced the speed of the outbreak.

3.15 Microsoft Security Patch

Because the incident was severe, Microsoft also released a security update for some older, unsupported Windows versions. This helped organizations whose older medical, industrial, or other systems could not be upgraded quickly.

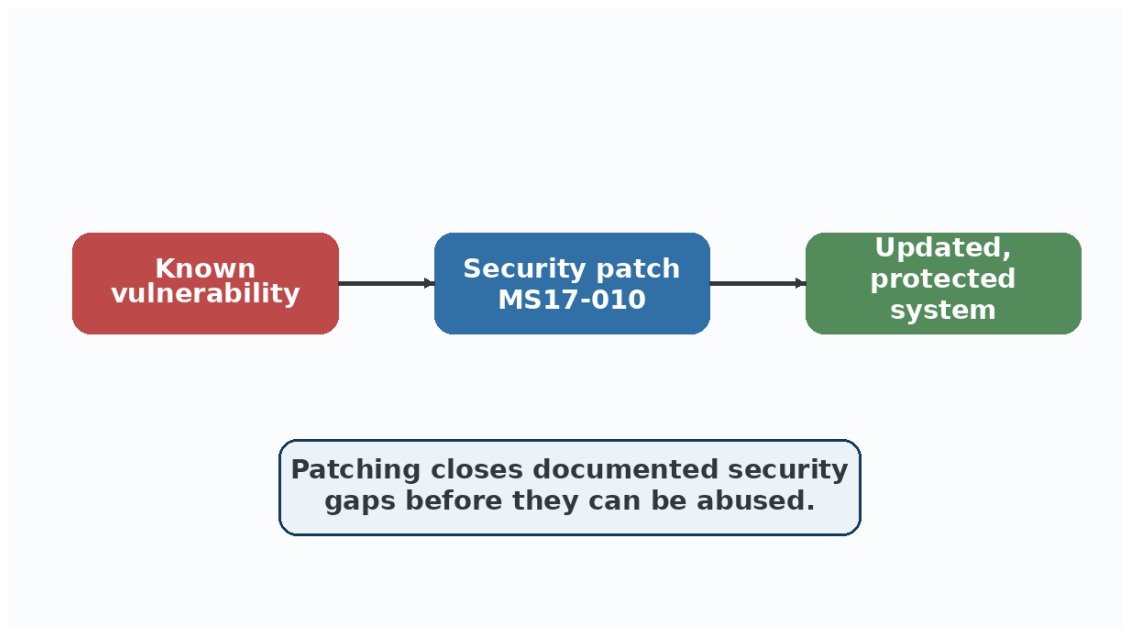


Figure 13. Installing Security Patches Closes Known Vulnerabilities

3.16 Lessons Learned

The main lesson from WannaCry is that releasing a patch is not enough; organizations must also install it. The outbreak also showed how one vulnerable computer can become a problem for a much larger network.

3.17 How Organizations Can Prevent Similar Attacks

Install security updates as soon as they are available, rather than delaying them.

Keep regular backups of important files, stored separately from the main network.

Limit which computers on a network can directly reach each other, so one infection cannot spread everywhere.

Train staff to recognise suspicious activity and report it quickly.

3.18 Importance of Operating System Updates

Operating system updates often include fixes for security flaws that have already been discovered, sometimes by the same researchers who help vendors like Microsoft patch them before attackers can widely exploit them. Delaying these updates leaves a known, documented weakness open, which is exactly what happened with the flaw behind WannaCry.

3.19 Summary

In summary, WannaCry became widespread because an existing Windows weakness remained unpatched on many systems. The case demonstrates the value of timely updates, good backups, network controls, and basic security awareness.

Case Study 4: Distributed Systems - GFS and Hadoop

4.1 Introduction

Some organizations handle data at a scale that is difficult for one computer to store or process. Distributed systems solve this problem by connecting many machines and sharing storage and computation. This case study focuses on Google's GFS and the open-source Hadoop platform.

4.2 What is a Distributed System?

A distributed system is a collection of independent computers that cooperate over a network and work like one overall system. The work and data are divided across the machines instead of being handled by a single large computer.

4.3 Why Distributed Systems are Used

They can store and process amounts of data too large for a single machine.

If one machine fails, the system can often keep working using the others.

New machines can be added over time to grow the system's total capacity.

Using many cheaper, ordinary machines can be more cost effective than one extremely powerful machine.

4.4 Google File System (GFS)

Google File System (GFS) is a distributed storage system created by Google for very large data sets. It was designed for Google's internal workloads and uses many ordinary machines instead of one very large storage server.

4.5 Why Google Developed GFS

Traditional single-computer file systems were not enough for Google's scale. With many machines running at once, hardware failure was expected, so GFS was designed to continue working when individual machines fail.

4.6 GFS Architecture

GFS splits large files into fixed-size chunks, usually 64 megabytes each, and spreads these chunks across many machines called chunkservers. A separate machine called the Master keeps track of where every chunk is stored, without holding the actual file data itself.

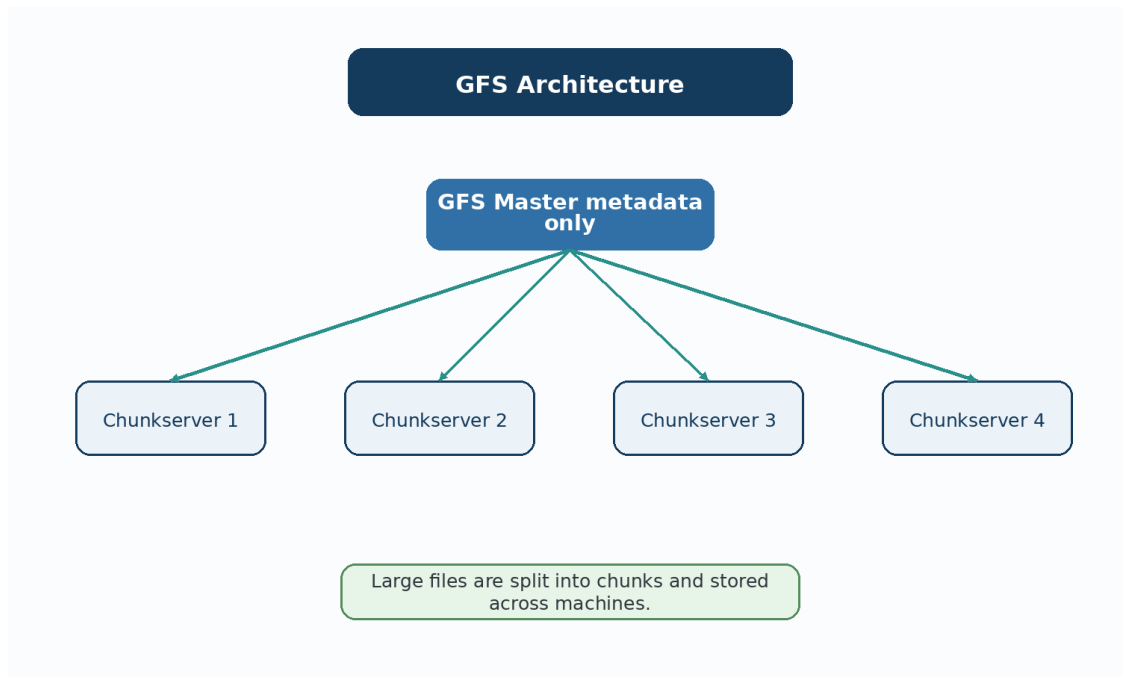


Figure 14. Google File System (GFS) Architecture

4.7 GFS Master

The GFS Master keeps metadata about files and chunks, including where each chunk is stored. It does not hold the actual file content, which keeps its main job focused on management and tracking.

4.8 GFS Chunkservers

Chunkservers are the machines that actually store the file data, broken up into chunks. A single file's chunks may be spread across many different chunkservers, letting GFS read or write different parts of a huge file at the same time using multiple machines.

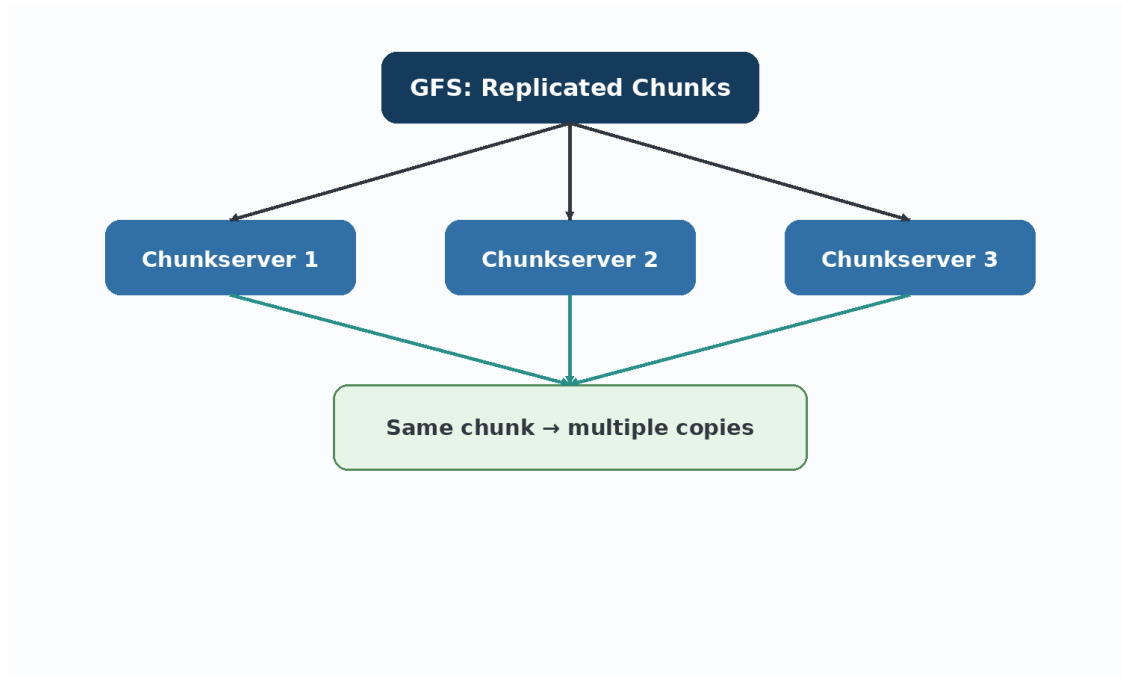


Figure 15. How the GFS Master Tracks Chunks Stored on Chunkservers

4.9 Data Replication

GFS improves reliability by keeping several copies of each chunk on different chunkservers, usually three. If one machine fails, another copy can still provide the data.

4.10 Fault Tolerance

Since GFS was built expecting hardware to fail regularly, it includes automatic checks that detect when a chunkserver goes offline or a chunk becomes corrupted. When this happens, GFS automatically creates new copies of the affected chunks on other healthy chunkservers, without needing a person to step in.

4.11 Hadoop

Hadoop is an open-source platform that adopted similar ideas for large-scale storage and processing. Unlike GFS, Hadoop is available for general use and has been adopted by many organizations.

4.12 Hadoop Distributed File System (HDFS)

HDFS is the storage part of Hadoop, and it follows a very similar design to GFS. Files are broken into large blocks and spread across many machines, with a central component keeping track of where everything is stored.

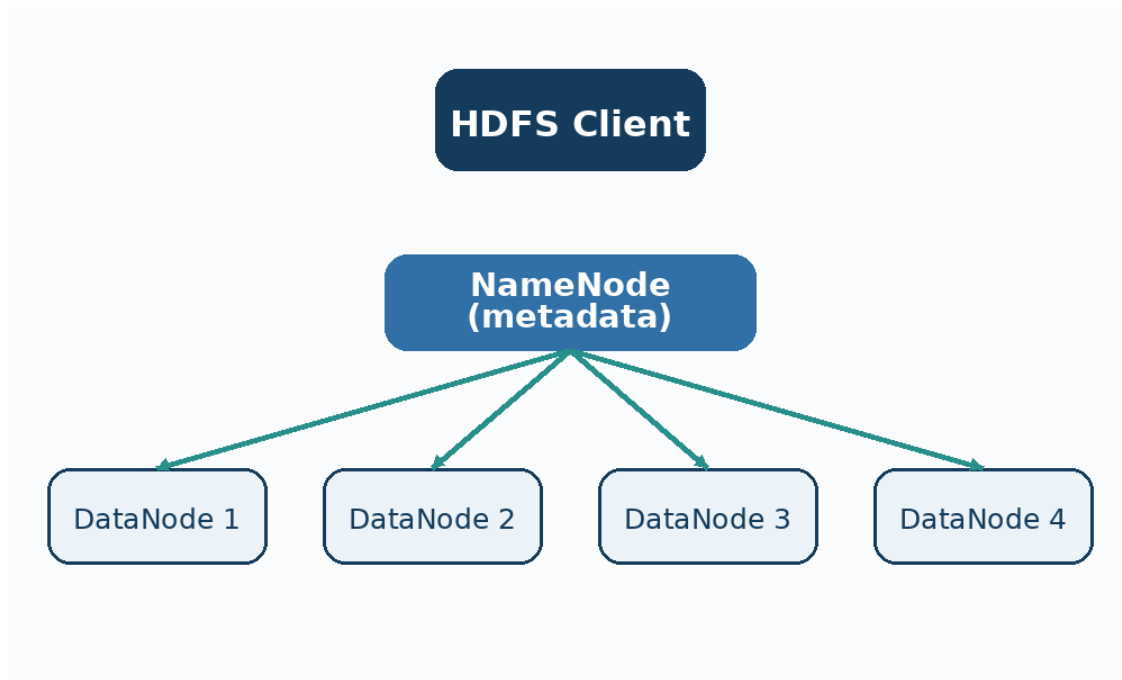


Figure 16. Hadoop Distributed File System (HDFS) Architecture

4.13 HDFS Architecture

HDFS uses two main types of machines: a NameNode, which keeps track of file locations, similar to the GFS Master, and multiple DataNodes, which store the actual file data blocks, similar to GFS chunkservers.

4.14 NameNode

The NameNode holds the metadata for the whole file system: file names, folder structure, and which DataNodes hold which blocks of each file. Like the GFS Master, it does not store the actual file content.

4.15 DataNodes

DataNodes are the worker machines in a Hadoop cluster that actually hold the file data. A large file is split into blocks, and those blocks are distributed across many DataNodes, letting Hadoop read and process the file in parallel using several machines at once.

4.16 Data Replication in HDFS

Just like GFS, HDFS keeps multiple copies of each data block, by default three, spread across different DataNodes. This means that if one machine in the cluster fails, the data it held is still safely available from other machines.

4.17 Hadoop MapReduce

MapReduce divides a large processing job into smaller tasks. The Map stage handles pieces of the input, and the Reduce stage combines the intermediate results. Running these tasks on multiple machines makes large data processing practical.

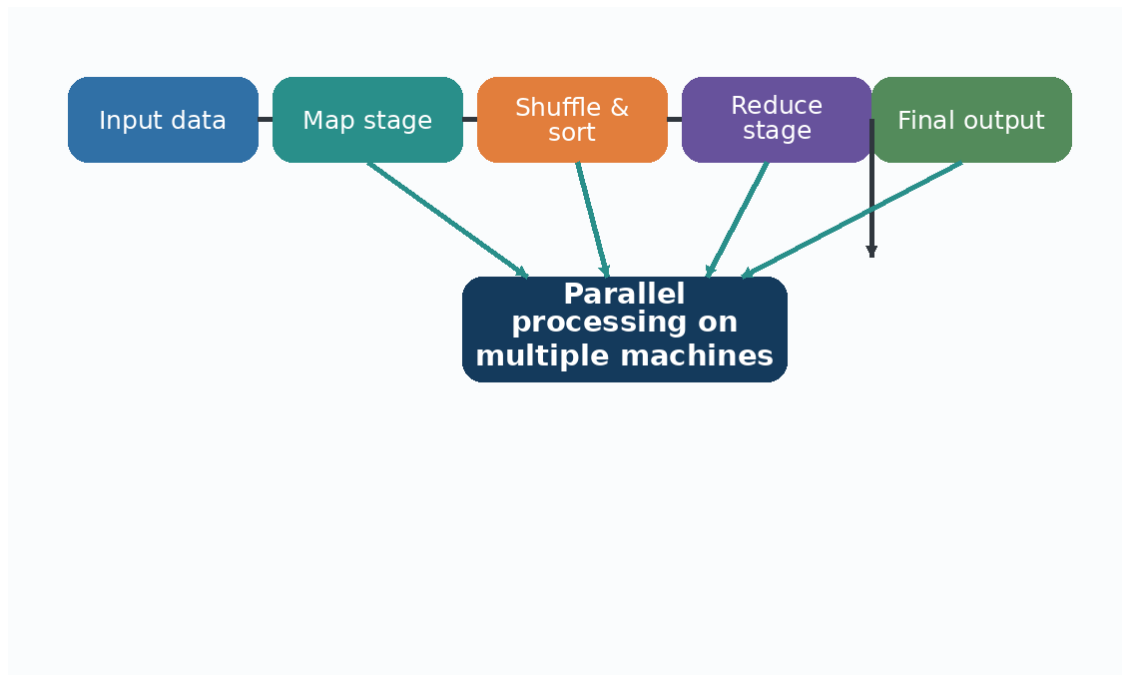


Figure 17. A Simplified MapReduce Workflow

4.18 Fault Tolerance in Hadoop

Hadoop is designed to expect machine failures as a normal part of running a large cluster. If a worker node fails during a MapReduce job, Hadoop automatically reassigns that piece of work to another healthy machine, so the overall job can still finish without a person needing to restart everything manually.

4.19 Real-World Applications

GFS supports many of Google's own services behind the scenes, including search indexing. Hadoop, being open source, has been adopted by a wide range of companies for tasks like analysing website traffic logs, processing large customer data sets, and running big data analytics pipelines.

4.20 GFS vs Hadoop

Feature	Google GFS	Hadoop
Purpose	Internal storage system built for Google's own services	General-purpose big data platform for many organisations
Architecture	Master and Chunkservers	NameNode and DataNodes (HDFS)
Storage	Proprietary, used only inside Google	Open source (HDFS), used by any organisation
Data Replication	Yes, typically 3 copies per chunk	Yes, typically 3 copies per block
Fault Tolerance	Automatic chunk recovery	Automatic block recovery and job retry
Processing	Paired with Google's internal processing tools	Paired with MapReduce and other tools like Spark

Feature	Google GFS	Hadoop
Scalability	Scales across Google's global data centers	Scales across any cluster of commodity machines
Typical Use	Google search and internal services	Business analytics, log processing, big data research

4.21 Advantages

Both systems can store and process data far beyond what a single machine could handle.

Automatic replication keeps data safe even when individual machines fail.

New machines can be added to grow storage and processing power over time.

Hadoop's open source nature makes this kind of large scale system available to any organisation, not just large tech companies.

4.22 Challenges

Running and maintaining a large cluster of machines takes real technical skill and effort.

Network communication between machines can become a bottleneck if not designed carefully.

Replicating data multiple times uses significantly more storage space overall.

Coordinating so many machines adds complexity that a single-computer system never has to deal with.

4.23 Summary

GFS and Hadoop both use many machines to store and process large amounts of data. GFS was created for Google's internal environment, while Hadoop made similar ideas widely available through open-source software. Their designs show the importance of replication, parallel processing, and fault tolerance in large systems.

References

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, Operating System Concepts, 10th ed. Wiley, 2018.
- [2] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Pearson, 2015.
- [3] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Pearson, 2018.
- [4] The Linux Kernel Organization, “The Linux kernel documentation.” [Online]. Available:
- [5] The Linux Kernel Organization, “The Linux kernel archives.” [Online]. Available:
- [6] Microsoft, “NTFS overview,” Microsoft Learn. [Online]. Available:
- [7] Apple Inc., “File system programming guide,” Apple Developer Documentation. [Online]. Available:
- [8] Microsoft Security Response Center, “Customer guidance for WannaCrypt attacks,” 2017. [Online]. Available:
- [9] Cybersecurity and Infrastructure Security Agency, “Ransomware guidance and resources.” [Online]. Available:
- [10] National Institute of Standards and Technology, “Cybersecurity framework resources.” [Online]. Available:
- [11] Microsoft, “Security update MS17-010 for Windows SMB server,” Microsoft Security Bulletin, 2017. [Online]. Available: